

# Annexe A15 — Le chantier Clear

LibraryV3 · Phase G, étape 2 · de la Référence J à la Référence K · 5 au 7 octobre 2026

Cette annexe historise l'étape 2 du plan de la phase G dans l'ordre où elle a été menée. Comme A14 et A14a, elle se lit comme une leçon : ce qu'on a supposé, ce qu'on a mesuré, ce qui a été réfuté, et ce qu'il faut en retenir. Elle répond aussi à une question qui surprend : **pourquoi des stores SSE2 de 16 octets battent-ils, pour l'instant, l'AVX2 sur une machine qui sait faire de l'AVX2 ?**

**Résultat en une ligne.** Clear passe de 1,25–1,28 ms à 0,46–0,55 ms ( $\div 2,4$ ), le budget moteur (Frame – Present) d'environ 2,05 à 1,20–1,34 ms ( $\times 0,6$ ), la frame de  $\approx 330$  à  $\approx 420$  fps. Cause : les effacements **attendaient** la mémoire (RFO et loi de Little), ils ne manquaient pas de largeur. Remède : des stores **non temporels**, en SSE2, sans seuil de taille. La largeur AVX2 n'apporte rien à fréquence nominale.

## 1. Le point de départ

Après le LOD des sphères (A14a), la Référence J laissait un constat net : Render ne pesait plus que 19 % de la frame, et Clear 42 %, plus du double. La campagne C2 l'avait déjà signalé sans l'expliquer : « environ 11 Mo effacés en 970  $\mu$ s, soit environ 11 Go/s, bien en deçà de ce qu'un memset vectorisé atteint ».

Clear ne dépend que de la résolution :  $1536 \times 900 \times 4$  octets = **5 529 600 octets** par buffer, deux buffers, soit 11,06 Mo par frame. Aucun LOD ne le réduira.

**Pourquoi deux effacements ?** Parce que la frame a deux mémoires, qui répondent à deux questions différentes. Le **buffer couleur** (dans la texture SDL, côté EXE) contient l'image : tout pixel qu'aucun triangle ne couvre doit montrer le fond, et le contenu de la texture verrouillée n'est pas garanti d'une frame à l'autre. Le **buffer de profondeur** (notre `std::vector<float>`, côté LIB) mémorise, pixel par pixel, la profondeur du plus proche triangle déjà dessiné ; en reverse-Z il repart de 0 (= infini). Sans cet effacement, TestAndSet comparerait chaque fragment aux profondeurs de la frame précédente : un objet qui s'éloigne resterait masqué par son propre fantôme. Les deux buffers ont des propriétaires différents, donc deux fonctions (`FrameBuffer::Clear`, `DepthBuffer::Clear`), et, on le verra, deux instructions machine différentes. Les quatre vues partagent ces deux buffers : un seul effacement de chaque par frame.

## 2. Étape 2a — Instrumenter avant d'optimiser

### 2.1 Découper la zone sans casser les références

La zone Clear mesurait deux opérations sans rapport : `Clean_Render` (EXE) et `Renderer::BeginFrame` → `DepthBuffer::Clear` (LIB). On a ajouté deux **sous-zones**, `ClearColor` et `ClearDepth`, placées **en fin** d'énumération pour que les colonnes des CSV de référence ne bougent pas, et **exclues du calcul de RESTE** (une sous-zone est déjà comptée dans sa mère : `kProfMainZoneCount`). La borne extérieure Clear n'a pas changé : elle reste comparable à J.

Deux compteurs temporaires visaient une troisième hypothèse : `ClearPageFaults` (défauts de page pendant Clear, lus **hors chronomètre**) et `LockAddrChanged` (SDL rend-il une autre mémoire à chaque frame ?). Le pilote SDL est désormais écrit dans l'en-tête des CSV (`sd1=`).

## 2.2 Trois hypothèses, chiffrées avant la mesure

| Hypothèse        | Mécanisme supposé                                                                | Prédiction                                                   |
|------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------|
| H1 symétrie      | les deux buffers s'écrivent de la même façon, avec RFO                           | $\text{ClearDepth} \div \text{ClearColor} \in [0,85 ; 1,15]$ |
| H2 chemins       | <code>fill(0.0f) → memset sans RFO ; fill(couleur) → boucle SSE2 avec RFO</code> | rapport < 0,75                                               |
| H3 texture neuve | SDL rend une mémoire jamais touchée : défauts de page au premier accès           | $\text{ClearPageFaults} \approx 1\,350$ par frame            |

## 2.3 Lot M14 : H3 réfutée par un contrôle positif, H2 réfutée par les chiffres

Médianes sur trois runs : ClearColor 0,65–0,66 ms, ClearDepth 0,60–0,62 ms, rapport frame par frame **0,91–0,92** ; ClearPageFaults = 0, LockAddrChanged = 0.

Un compteur qui affiche toujours zéro ne prouve rien : il peut être débranché. Les séries brutes contenaient la preuve qu'il fonctionnait : **frame 0 (chauffe) : 1 350 défauts de page et LockAddrChanged = 1**, exactement  $5\,529\,600 \div 4\,096$ . C'est le premier contact avec la texture staging que Direct3D 9 crée au premier verrouillage ; ensuite, SDL rend toujours la même mémoire. **H3 est réfutée**, avec un témoin qui a vu le phénomène une fois, au bon endroit, avec la bonne amplitude.

H2 attendait un rapport < 0,75 ; on mesure 0,91. **H2 est réfutée**. H1 est retenue sous réserve : les deux buffers plafonnent au même débit, environ 8,4 à 9,2 Go/s.

**À retenir.** On découpe une zone avant d'y toucher, sans casser ni les colonnes de référence ni le calcul du reste. Tout compteur nouveau doit avoir vu au moins une fois le phénomène qu'il prétend exclure : c'est son contrôle positif.

## 3. Lire le code machine qui tourne vraiment

### 3.1 Quatre pièges du débogueur

- **Lire le Debug pour juger le Release.** Le premier désassemblage venait d'un build Debug : `__CheckForDebuggerJustMyCode`, `_RTC_CheckStackVars`, aucun inlining. Le rep `stos dword ptr [rdi]` qu'on y voyait remplissait la **pile** de `0xCC` (104 octets, /RTCs) : ce n'était pas l'effacement. Le prendre pour le rep `stosb` de H2 aurait validé une hypothèse sur un artefact.
- **Croire que le Release ne se débogue pas.** /Zi et /DEBUG étaient déjà actifs ; un .pdb ne change pas le code machine. Il suffit de lancer par F5, pas par Ctrl+F5 ni par le .bat.
- **Chercher « le code de la LIB ».** `FrameBuffer::Clear`, `DepthBuffer::Clear` et `BeginFrame` sont définies dans des en-têtes : elles sont compilées dans `main.cpp`, puis inlinées par le LTCG, jusqu'à main lui-même, recopié dans `invoke_main`. La pile des appels n'affichait que `invoke_main`, et F11 dans l'éditeur source sautait tout l'effacement d'un coup.
- **Dépendre de la fenêtre Registres.** Elle peut rester vide ; la fenêtre Espion accepte r8 et des expressions comme `*(unsigned __int64*)0x7FF84FCC6090`.

La méthode qui a fonctionné : une **sonde de lecture** `__declspec(noinline) + __debugbreak()` dans la fonction visée, sur un build séparé, retirée ensuite. Elle redonne une vraie fonction au débogueur sans changer la boucle chaude.

### 3.2 Ce que fait vraiment le code

| Buffer     | Code source                                      | Instruction exécutée                          | Appels par frame              |
|------------|--------------------------------------------------|-----------------------------------------------|-------------------------------|
| Couleur    | <code>std::fill(row, row + w, 0xFF000018)</code> | <code>rep stos dword ptr [rdi]</code>         | $900 \times 6\,144$ octets    |
| Profondeur | <code>std::fill(begin, end, 0.0f)</code>         | <code>memset → rep stos byte ptr [rdi]</code> | $1 \times 5\,529\,600$ octets |

La STL de MSVC détecte elle-même une valeur dont tous les bits sont nuls et appelle `memset` (`test ebx,ebx / jne`). Le `memset` du CRT choisit son chemin à l'exécution : seuil bas `[6090h] = 32\,768` octets, bit ERMS levé `[6374h] = 2`, et **seuil des stores non temporels** `[6098h] =  $2^{64} - 1$`  : dans cette version du CRT et sur cette machine, ce chemin est désactivé. Pour 5,5 Mo, on part en `rep stosb`.

Conséquences : aucune boucle vectorisée compilée par nous n'intervient (`/arch` ne joue aucun rôle), les deux buffers passent par le **microcode des instructions de chaîne**, et les 9 % d'écart entre couleur et profondeur s'expliquent par 900 démarrages de `rep stosd` contre un seul `rep stosb`.

**X Un point d'arrêt dans une fonction de bibliothèque attrape tous ses appelants.** Les arrêts suivants dans `memset (vmovdqa)` venaient d'autres appels de moins de 32 Ko, probablement le `ProfFrame{}` du profileur. On vérifie toujours à qui appartient un arrêt (ici : `r8 = 5\,529\,600`).

## 4. Étape 2b — Encadrer la machine

### 4.1 Trois façons d'écrire les mêmes octets

La question de fond restait ouverte : le microcode de `rep stos` écrit-il des lignes de cache entières **sans les lire** (sans RFO, *Read For Ownership*) ? Le désassemblage ne peut pas répondre : la même instruction peut éviter le RFO ou non selon le processeur. On a donc encadré la machine par deux bornes connues :

| Variante        | Écriture                                                        | RFO                         |
|-----------------|-----------------------------------------------------------------|-----------------------------|
| V0 STL          | <code>rep stosd / rep stosb</code> , le code de J à l'identique | inconnu : c'est la question |
| V1 ordinaire    | <code>_mm_store_si128</code> , 64 octets par tour               | oui, en principe            |
| V2 non temporel | <code>_mm_stream_si128</code> , puis <code>sfence</code>        | non                         |

Prédictions :  $\alpha$  ( $V1 > 1,25 \times V0$ ,  $V2 \approx V0$  : `rep stos` évite déjà le RFO),  $\beta$  ( $V1 \approx V0$ ,  $V2 < 0,75 \times V0$  : `rep stos` paie le RFO),  $\gamma$  (tout égal à  $\pm 12\%$  : le RFO ne limite pas).

Vérification au débogueur avant le banc : V1 donne bien quatre `movdqu` par tour (`movdqu` aligné vaut `movdqa` depuis Nehalem), V2 quatre `movntdq` et un `sfence`. Le compilateur avait transformé la **queue scalaire** en `rep stosd` ; des points d'arrêt sur la tête et la queue ont prouvé qu'elles ne s'exécutaient jamais (lignes SDL alignées sur 16 octets, 6 144 octets = 96 lignes de cache).

**X Une variante qu'on n'a pas vérifiée n'est pas une variante.** Le compilateur reconnaît les boucles de remplissage et les remplace par `memset` ou `rep stos`. On l'a vu trois fois dans ce chantier.

### 4.2 Lot M15 (1536 × 900) : $\beta$ , puis au-delà

| Grandeur        | $V1 \div V0$ (a / b) | $V2 \div V0$ (a / b) |
|-----------------|----------------------|----------------------|
| ClearColor      | 1,08 / 1,07          | <b>0,36 / 0,39</b>   |
| ClearDepth      | 1,25 / 1,13          | <b>0,48 / 0,45</b>   |
| Render (témoin) | 0,94 / 1,03          | 0,85 / 0,87          |
| Budget moteur   | 1,05 / 1,07          | <b>0,60 / 0,61</b>   |

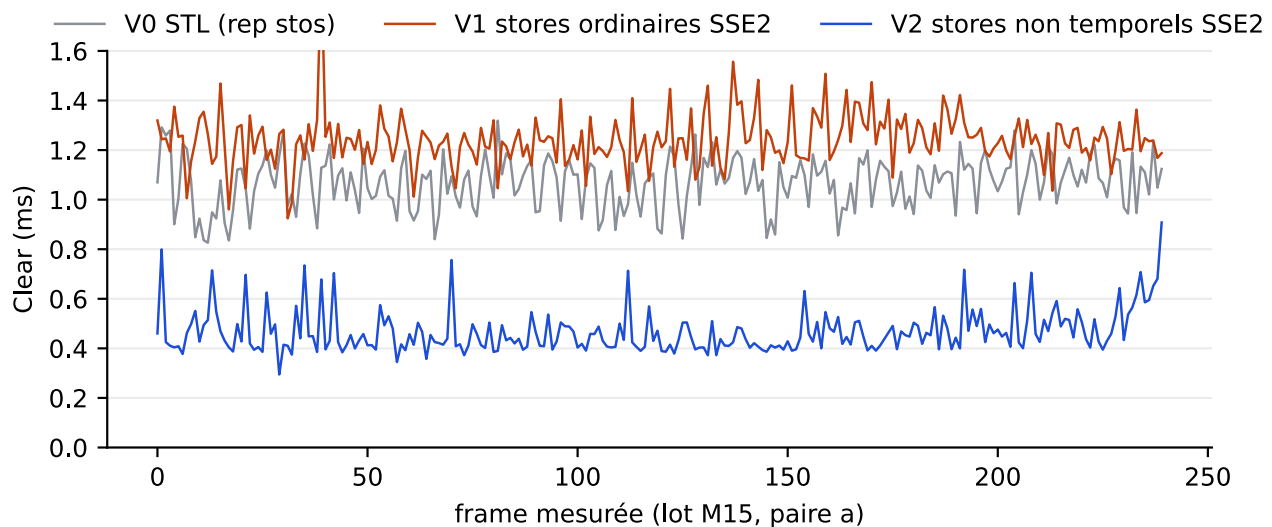


Figure 1 — Clear frame par frame, paire a du lot M15. V0 et V1 se superposent vers 1,1 à 1,2 ms ; V2 tombe vers 0,45 ms. Les trois binaires calculent la même scène (compteurs identiques).

V1  $\approx$  V0 : rep stosd se comporte comme des stores ordinaires,  **$\alpha$  est réfutée**. V2 très en dessous de 0,75 :  **$\beta$  est confirmée**. Mais le facteur ne colle pas : supprimer le RFO divise les octets transportés par deux, le rapport ne devrait pas descendre sous 0,5. On mesure 0,36 pour la couleur, soit environ 25 Go/s contre 8,4. Le modèle « on paie en octets » était incomplet.

#### 4.3 L'explication : on payait en attente (loi de Little)

Un cœur ne garde en vol qu'un nombre limité de lignes qu'il attend de la mémoire : les tampons de remplissage du L1, environ 16 sur un cœur P d'Alder Lake. La loi de Little relie trois grandeurs :

$$\begin{aligned} \text{débit} &= (\text{requêtes en vol} \times \text{taille d'une requête}) \div \text{latence} \\ &= 16 \times 64 \text{ octets} \div \sim 110 \text{ ns} \approx 9,3 \text{ Go/s} \end{aligned}$$

Avec un store ordinaire, chaque ligne doit d'abord être **lue** en DRAM (RFO) et occupe un tampon pendant tout l'aller-retour. Le cœur ne sature pas les octets : il **attend**, avec seulement 16 lignes en vol. C'est l'ordre de grandeur du plateau mesuré pour V0 et V1 : 8,4 à 9,2 Go/s pour la couleur selon les lots ; le rep stosb d'un seul bloc du depth fait un peu mieux (9 à 11 Go/s), sans changer de famille. Avec un store non temporel, les écritures se regroupent en lignes complètes dans des tampons de combinaison d'écriture et partent vers la mémoire **sans attendre de réponse** : plus d'aller-retour, plus de limite par la latence. Les valeurs 16 et 110 ns sont des ordres de grandeur, non mesurés ; mais le modèle reproduit le plateau chiffré.

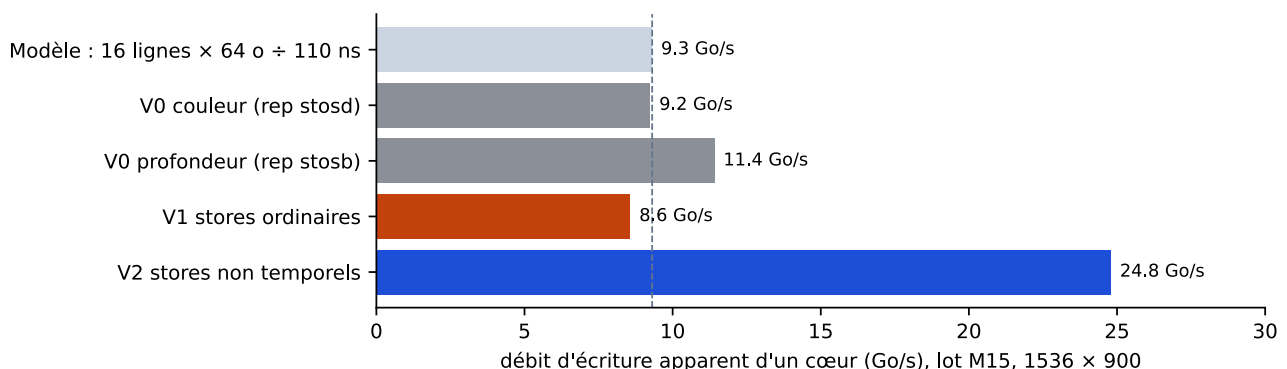


Figure 2 — Débit apparent par variante (lot M15). Les stores qui passent par le cache restent au voisinage de la droite de Little (9 à 11 Go/s) ; seuls les stores non temporels s'en affranchissent, avec un facteur 2,2 à 2,8.

#### 4.4 Le témoin Render baisse : un coût déplacé à notre avantage

On prédisait une hausse de Render inférieure à 0,05 ms avec V2 (le depth n'est plus en cache). On mesure une **baisse** de 13 à 15 %. Explication la plus plausible : V0 laissait environ 11 Mo de lignes **modifiées** dans les caches (L3 de 12 Mo) ; chaque donnée chargée ensuite par Render évinçait une ligne modifiée, réécrite en DRAM. Render payait une partie de la facture de Clear. Le vrai coût de V0 dépassait donc son 1,085 ms affiché ; seul le budget moteur le voit en entier (−0,74 ms contre −0,64 ms sur Clear seul).

#### 5. Lot M16 (768 × 450) : faut-il un seuil de taille ?

Microsoft a désactivé les stores non temporels dans memset parce qu'un memset générique ne sait pas si les données seront relues bientôt : si elles sont encore en cache, un store ordinaire y écrit sans aller-retour DRAM. On a donc mesuré une résolution où les deux buffers tiennent dans le L3 (2 × 1,38 Mo).

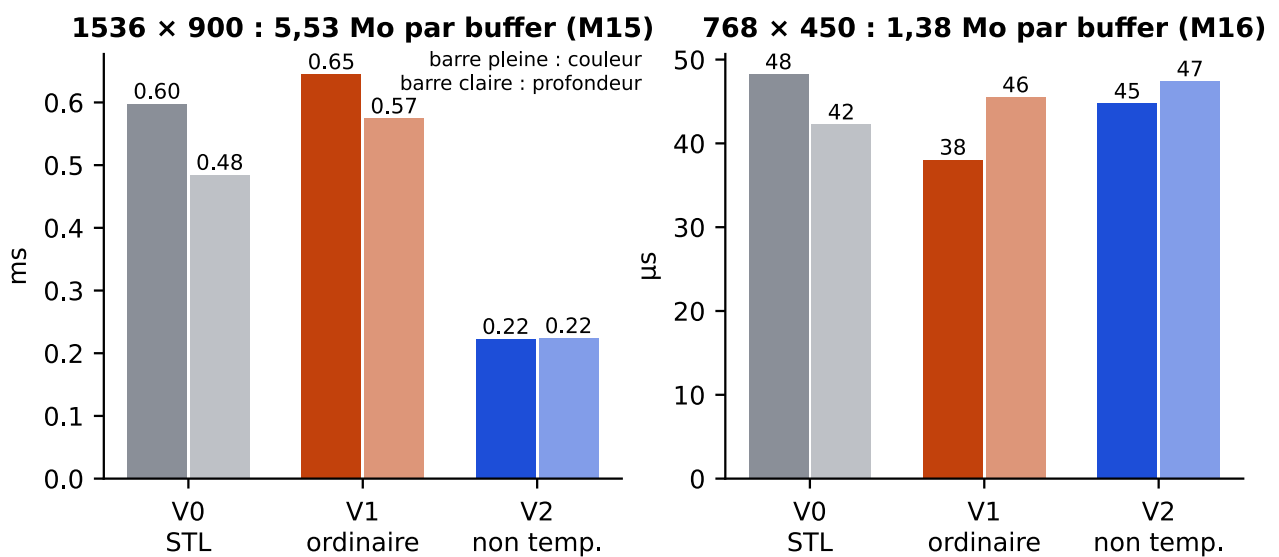


Figure 3 — Coût par buffer selon la variante. À 5,53 Mo, les stores non temporels divisent le coût par 2,2 à 2,8. À 1,38 Mo (buffers dans le L3), les trois variantes se valent.

Les trois prédictions tiennent ( $V2 \approx 0,09$  ms,  $V0 \leq 0,11$  ms, budget moteur  $V2 \div V0 = 0,99$  à 1,00), avec une nuance : on attendait que les stores non temporels **perdent** sur un petit buffer, ils font **jeu égal** (au plus +5 µs sur le depth). Un seuil aurait ajouté une branche, une constante et un cas de test sans gain mesurable : **un seul chemin**.

✗ **Garder « par prudence » les variantes et le commutateur dans le code livré.** Un commutateur de mesure qui reste dans le code finit toujours par être basculé par erreur. Le code livré n'a qu'un chemin, documenté par les lots qui l'ont choisi.

## 6. Pourquoi SSE2 et pas AVX2 (pour le moment)

C'est le résultat le plus contre-intuitif du chantier, et il tient en une phrase : **la largeur des registres n'accélère que le code limité par le calcul ; Clear est limité par la mémoire.** Voici le raisonnement, puis la mesure.

### 6.1 Le cœur sait déjà écrire plus vite que la mémoire n'absorbe

$5\,529\,600 \div 16 \text{ octets} = 345\,600 \text{ stores SSE2}$ . Même à un seul store par cycle et 4 GHz, le cœur les **émet** en 86  $\mu\text{s}$  ; la mémoire met environ 220  $\mu\text{s}$  à les **absorber**. Pendant plus de 60 % du temps, le cœur attend. L'AVX2 diviserait l'émission par deux (43  $\mu\text{s}$ ) : il raccourcirait une attente qui ne limite rien.

### 6.2 Ce qui part vers la mémoire, ce sont des lignes de 64 octets

Avec des stores non temporels, quatre stores SSE2 ou deux stores AVX2 remplissent le **même** tampon de 64 octets, qui part quand la ligne est complète. Le contrôleur mémoire reçoit exactement les mêmes transactions : la largeur du registre disparaît avant d'atteindre la DRAM.

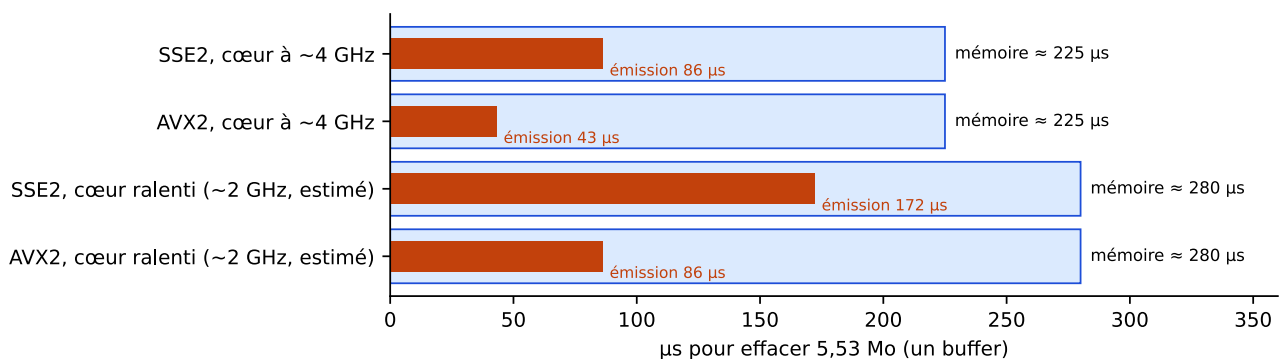


Figure 4 — Temps d'émission des stores par le cœur (orange, modèle à un store par cycle) face au temps d'absorption par la mémoire (bleu, mesuré). Tant que l'orange reste loin à gauche du bleu, élargir les registres ne change rien. Quand le cœur ralentit, l'émission se rapproche du goulot.

### 6.3 La mesure : variante V3 (\_mm256\_stream\_si256), deux lots

V3 ne change que la largeur : même non-temporalité, mêmes lignes, même ordre, intrinsèques seulement, **sans** /arch:AVX2. Deux pièges propres à l'AVX2 ont été traités : l'alignement sur 32 octets (rattrapé par un store SSE de 16 octets, jamais en scalaire, sinon le compilateur aurait créé un rep stosd par ligne), et la **transition AVX/SSE** (vzeroupper après le corps, car tout le reste du moteur est compilé en SSE). Le compilateur sauvegarde alors la constante 256 bits en pile et la recharge à chaque ligne. Contrôle visuel : fond uniforme, sans les rayures que produirait une moitié haute perdue.

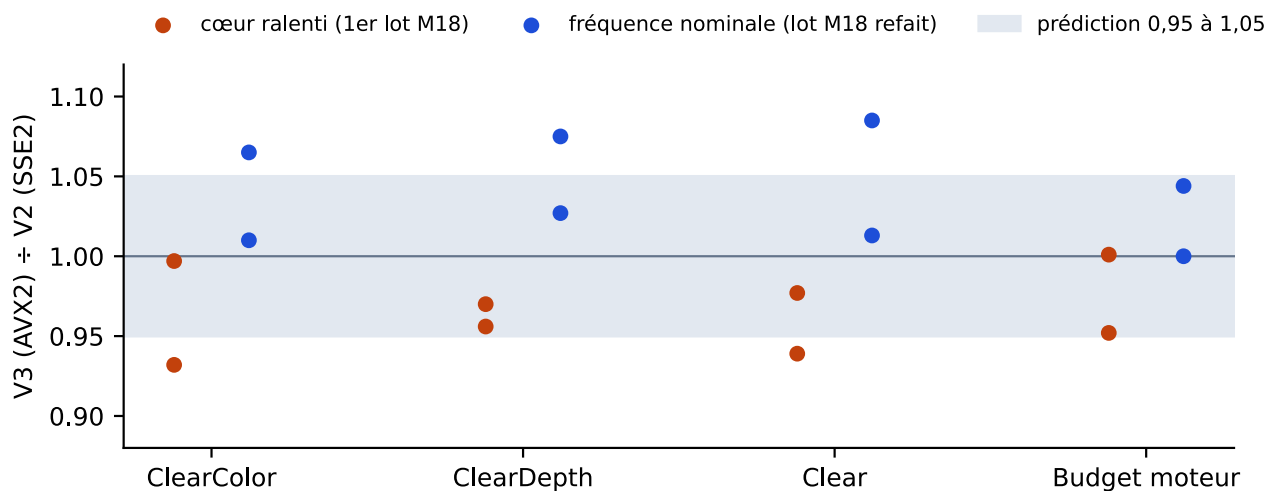


Figure 5 — Rapport  $V3 \div V2$  par paire, dans deux états de la machine. Cœur ralenti : léger avantage à l'AVX2 (–2 à –7 % selon la grandeur). Fréquence nominale : aucun avantage, voire un léger surcoût.

Le premier lot M18 a été mesuré pendant un épisode où le processeur tournait environ deux fois moins vite (overhead\_ns 60 à 72 au lieu de 32, Animation et Render  $\times 1,7$ , Clear  $\times 1,15$  seulement). L'émission SSE2 se rapprochait alors du goulot ( $\approx 170 \mu\text{s}$  sur 280), et diviser le nombre de stores aidait un peu. Refait à fréquence nominale, le lot donne  $V3 \div V2 = 1,01$  à  $1,09$  sur Clear : il ne reste que le surcoût propre de V3 (900 vzeroupper, 900 rechargements, le rattrapage d'alignement), minuscule mais plus compensé.

**À retenir.** « Limité par le calcul » ou « limité par la mémoire » n'est pas une propriété du code seul : c'est un **rapport** entre la vitesse du cœur et celle de la mémoire. Le test : comparer le temps mesuré au temps qu'il faudrait au cœur pour **émettre** les instructions. Grand écart : vectoriser ne sert à rien, il faut écrire moins ou supprimer les attentes.

## 6.4 La décision, et quand l'AVX2 reviendra

V2 (SSE2 non temporel) est retenue : elle est au moins aussi rapide à fréquence nominale et n'impose aucune contrainte manuelle. Les trois contraintes de V3 (vzeroupper à la main, constante en pile, alignement sur 32) **disparaîtront d'elles-mêmes** avec le chantier AVX2 : quand /arch:AVX2 sera posé pour tout le moteur, tout le code sera encodé en VEX et le compilateur placera seul les vzeroupper. Le noyau passera alors en 256 bits sans précaution, puis en AVX-512 sur l'i9, avec une détection CPUID au démarrage (l'i7-1265U n'a pas d'AVX-512 : instruction illégale sinon) et un alignement sur 64 octets.

L'AVX2 rapportera vraiment là où la largeur fait le débit : VertexStage::TransformPositions (« le point unique où vivra la version AVX2 »), les fonctions d'arête du rasterizer, et peut-être le coût fixe de Render, une fois qu'on saura s'il est fait de calcul ou d'attente.

**✗ Élargir les registres parce que la machine le permet.** Un code qui attend la mémoire ne va pas plus vite avec des registres plus larges ; il gagne seulement des contraintes. On mesure d'abord ce qui limite.

## 7. Une règle tirée du chantier : M19, le thermomètre de fréquence

Le profileur mesure à chaque run le coût d'une paire de bornes (overhead\_ns). Il a servi, sans qu'on l'ait prévu, de thermomètre de fréquence : c'est lui qui a révélé l'état dégradé du premier lot M18.

**M19.** Un run n'est recevable que si son overhead\_ns reste à  $\pm 30\%$  de celui du jetable du même lot, et ses zones limitées par le calcul (Animation, LocalXform1) à  $\pm 15\%$ . Sinon, l'état de la machine a changé : le lot est à refaire, ou recevable seulement pour des comparaisons internes déclarées.

Une seconde observation borne ce que M19 peut voir : à témoins de fréquence identiques (overhead\_ns = 32), V2 mesurait 0,445 ms un jour et 0,50 ms un autre. L'état de la mémoire et l'activité de fond de Windows ne se voient pas dans overhead\_ns : c'est le plancher de bruit de C2 (12 à 15 %). D'où la règle de fait appliquée depuis M13 : **toute comparaison se fait par paires A/B dans un même lot**, et une référence absolue se fixe par son propre lot.

## 8. La Référence K (7 octobre 2026)

Lot M17 : i7-1265U épinglé (affinity=4), sur secteur, binaire nettoyé (stores non temporels SSE2, plus de commutateur), scène solar\_system\_vlcompat\_belt\_lod\_spheres.json, 4 vues, 1536 × 900, Release. Jetable puis K1, K2, K3, tous recevables par M19.

| Grandeur                        | Référence K                | Part de la frame | Référence J                |
|---------------------------------|----------------------------|------------------|----------------------------|
| Clear                           | <b>0,46 à 0,55 ms</b>      | ≈ 21 %           | 1,25 à 1,28 ms             |
| dont ClearColor / ClearDepth    | 0,23 à 0,27 ms chacun      |                  | ≈ 0,63 ms chacun           |
| Render                          | 0,57 à 0,62 ms             | ≈ 25 %           | 0,57 à 0,59 ms             |
| Present (hors budget moteur)    | 1,03 à 1,11 ms             | ≈ 45 %           | 0,94 à 0,99 ms             |
| Budget moteur (Frame – Present) | <b>1,20 à 1,34 ms</b>      |                  | ≈ 2,05 ms                  |
| Frame                           | 2,24 à 2,48 ms (≈ 420 fps) |                  | 2,96 à 3,07 ms (≈ 330 fps) |
| FacesSubmitted                  | 14 752                     |                  | 14 752                     |

**Attention à une lecture trop rapide de ce tableau :** Render y paraît inchangé entre J et K, alors que le lot M15 mesurait  $-13$  à  $-15\%$  pour V2 face à V0 **dans le même lot**. Les deux sont vrais : entre J (5 octobre) et K (7 octobre), la machine a dérivé d'environ  $+12\%$  sur les zones de calcul à témoins de fréquence identiques (§ 7), ce qui masque le gain. Seule la comparaison par paires dans un lot mesure l'effet ; deux références de jours différents mesurent aussi la machine.

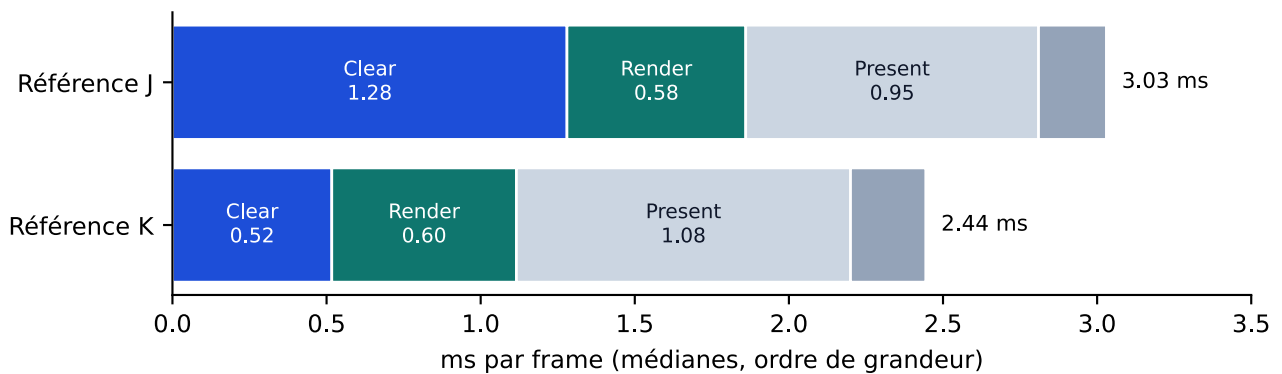


Figure 6 — Répartition de la frame (somme de médianes, ordre de grandeur). Clear a fondu ; Present, hors budget moteur, est devenu le plus gros poste.

## 9. Les contre-exemples, rassemblés

- ✗ Optimiser avant de découper : remplacer `std::fill` par des stores non temporels « parce que le banc isolé l'a montré » aurait été inutile si H3 avait été vraie (§ 2).
- ✗ Un compteur sans contrôle positif : zéro ne prouve rien tant qu'on n'a pas vu le compteur compter (§ 2.3).
- ✗ Lire l'assembleur du Debug pour juger le Release : le `rep stos` de /RTCs n'était pas l'effacement (§ 3.1).
- ✗ Croire au code source plutôt qu'au désassemblage : la STL, le CRT et le compilateur ont réécrit trois fois nos boucles (§ 3.2, § 4.1).
- ✗ Une seule variante face à la référence : sans la borne V1, un échec de V2 aurait été ininterprétable (§ 4.1).
- ✗ Prendre un coût affiché pour le coût réel : V0 faisait payer à Render l'éviction de ses lignes modifiées (§ 4.4).
- ✗ Garder les variantes de mesure dans le code livré (§ 5).
- ✗ Élargir les registres d'un code limité par la mémoire (§ 6).
- ✗ Comparer des runs sans vérifier l'état de la machine : un cœur ralenti avait inversé le verdict AVX2 (§ 6.3, § 7).

## 10. Ouvert

| Point                                                                                                                                | Où               | Quand                  |
|--------------------------------------------------------------------------------------------------------------------------------------|------------------|------------------------|
| Dettes Serializer (clés optionnelles, surcharges NaN) et cache de <code>LoadMeshChecked</code>                                       | LIB              | étape 3, maintenant    |
| Nettoyage : <code>Meshesrock_b/c.lod.json</code> , clés « _ » de <code>gen_rock_lod.py</code>                                        | dépôts           | étape 3                |
| Décomposer le coût fixe de Render ( $\approx 0,39$ ms) par ablation                                                                  | RenderView       | après l'étape 3        |
| Present ( $\approx 45$ % de la frame) : D3D9 <code>UpdateTexture</code> de 5,5 Mo ; pilote D3D11, mode de mise à jour, double tampon | EXE / SDL        | chantier futur         |
| Chantier AVX2 : <code>/arch:AVX2</code> pour tout le moteur, nouvelle référence ; noyau d'effacement en 256 bits                     | LV3.Common.props | chantier futur         |
| Variante AVX-512 sur l'i9 : détection CPUID, alignement 64 octets                                                                    | LIB              | après le chantier AVX2 |
| Comparer les deux machines (i9-10900X : mémoire 4 canaux, cœur seul souvent plus lent en débit)                                      | banc             | à l'occasion           |

## 11. Aide-mémoire

| Grandeur ou règle                                           | Valeur                                                                                                |
|-------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Taille d'un buffer plein écran                              | $1536 \times 900 \times 4 = 5\,529\,600$ octets                                                       |
| Plateau des stores passant par le cache (un cœur, i7-1265U) | $\approx 9$ à $11$ Go/s ; modèle $\sim 16$ lignes $\times 64$ o $\div \sim 110$ ns $\approx 9,3$ Go/s |
| Stores non temporels (un cœur)                              | $\approx 25$ Go/s à $1536 \times 900$ ; $\approx 30$ Go/s à $768 \times 450$                          |
| memset du CRT sur cette machine                             | rep stosb au-delà de 32 Ko ; chemin non temporel désactivé                                            |
| Code livré                                                  | ClearKernel::StreamFill (SSE2 _mm_stream_si128) + Fence() une fois par buffer                         |
| Test « calcul ou mémoire ? »                                | temps mesuré $\div$ temps d'émission des instructions ; $\gg 1$ : mémoire                             |
| M19                                                         | overhead_ns $\pm 30$ %, Animation / LocalXform1 $\pm 15$ % du jetable                                 |
| Prochaine étape                                             | étape 3 (dettes), puis le coût fixe de Render                                                         |